IAM-05: Boundary Design Patterns

> **Series:** IAM — IAM Administration | **Notebook:** 5 of 12 | **Created:** January 2026 | **Last Updated:** 04/30/2026

Controlling Data Visibility with Boundaries

Boundaries determine **what data** users can see. While policies control actions, boundaries filter visibility. This notebook covers boundary syntax, patterns, and implementation strategies.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Gen3 IAM enabled
Permissions	`environment-admin` or boundary management rights

Requirement	Details
Prior Knowledge	**IAM-01** through **IAM-04**

1. Boundary Fundamentals

Boundaries filter what entities and data a user can see within an environment.

Policies vs Boundaries

Concept	Controls	Example
Policy	What actions	"Can read logs"
Boundary	What data	"Logs from checkout service only"

A user needs BOTH:
- A **policy** granting the action (e.g., `storage:logs:read`)
- A **boundary** including the data (e.g., `checkout` security context)

How Boundaries Work

1. User attempts to access data (query, view, etc.)
2. Dynatrace checks if user's policy allows the action
3. Dynatrace filters results to match user's boundary
4. User sees only data within their boundary

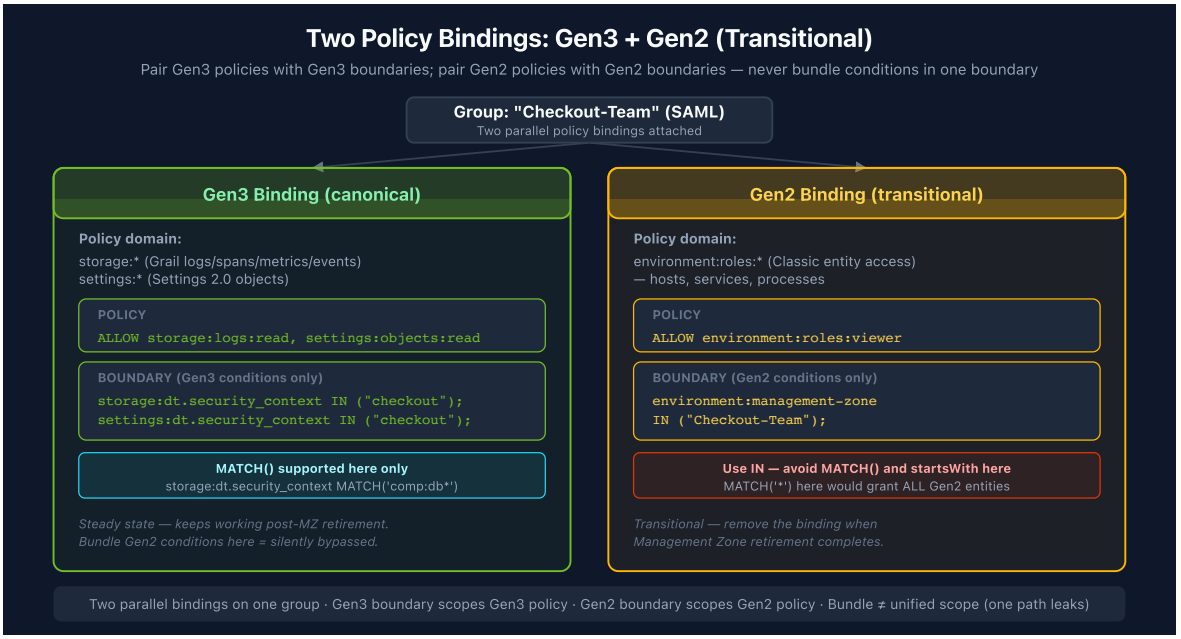
Boundary Scope

Boundaries are **environment-level** and assigned to groups:

```
Group: dt-checkout-editors
├── Policy: environment-editor
└── Boundary: checkout-services-only
```

2. The Three-Domain Model

Boundaries filter across three domains. A complete boundary typically includes all three.



Domain 1: Environment

Controls visibility of Smartscape entities:
- Hosts
- Services
- Applications
- Databases
- Cloud resources

Domain 2: Storage

Controls visibility of Grail data:
- Logs
- Spans (traces)
- Metrics
- Events
- Business events

Domain 3: Settings

Controls visibility of configuration:
- Alerting rules
- SLOs
- Custom settings

Why All Three Matter

If you only boundary one domain:

Missing Domain	Problem
Environment	User can't see services in UI
Storage	User can't query logs/spans
Settings	User can't see/edit config

3. Boundary Syntax Reference

Basic Boundary Structure

```
&lt;domain&gt;:&lt;field&gt; &lt;operator&gt; (&lt;values&gt;)
```

Pair Gen2 policies with Gen2 boundaries; pair Gen3 policies with Gen3 boundaries. Each boundary's conditions evaluate against the policy it's attached to — using `environment:management-zone` to scope a `storage:logs:read` policy is a no-op (the Gen3 policy doesn't read `environment:` conditions), and using `storage:dt.security_context` to scope an `environment:roles:viewer` policy is also a no-op (Classic entity access doesn't read Gen3 conditions). Mixing them in one boundary creates the illusion of unified scope while leaving one path wide open.

The right shape is **two parallel policy bindings on the same group**:

```
# Gen3 policy
ALLOW storage:logs:read,
      storage:spans:read,
      storage:metrics:read,
      settings:objects:read,
      settings:objects:write;
```

```
# Gen3 boundary
storage:dt.security_context  IN ("checkout");
settings:dt.security_context IN ("checkout");
```

```
# Gen2 policy (transitional, for Classic entity access)
ALLOW environment:roles:viewer;
```

```
# Gen2 boundary
environment:management-zone IN ("checkout");
```

Attach both to the user's group. When Management Zone retirement completes, the Gen2 binding is removed cleanly without touching the Gen3 one. **A `MATCH` operator on a Gen2 policy's boundary (e.g., `MATCH('*')` against `storage:entities:read`) would grant access to all Gen2 entities — `MATCH` belongs in Gen3 boundaries only.**

Operators

Operator	Description	Example
`IN`	Match any in list	`IN ("a", "b", "c")`
`=`	Exact match	`= "checkout"`
`!=`	Not equal	`!= "restricted"`
`startsWith`	Prefix match	`startsWith "team-"`
`contains`	Substring	`contains "prod"`
`MATCH`	Wildcard pattern match	`storage:dt.security_context MATCH('*/app:easytrade')`

> **`MATCH()` is available for the `storage` and `settings` domains.** It supports `*` as a wildcard at any position — anchor with a trailing `*` to mimic prefix matching.

> The `environment:` domain (Classic Management Zones) does **not** support `MATCH()` — use `IN` (preferred) or `startsWith` there.
>
> ⚠️ **`storage:smartscape:read` + `startsWith` has a known bug.** Use `MATCH('value*')` instead (e.g. `MATCH('comp:db*')`) for Smartscape and Classic entity access.

Common Fields

Domain	Field	Description
environment	`management-zone`	Management zone filter
storage	`dt.security_context`	Data security context
storage	`bucket-name`	Storage bucket name
settings	`dt.security_context`	Settings security context
settings	`schemaId`	Settings schema

Multiple Values

Grant access to multiple security contexts (two parallel policy bindings):

```
# Gen3 policy
ALLOW storage:logs:read,
      storage:spans:read,
      settings:objects:read;
```

```
# Gen3 boundary
storage:dt.security_context  IN ("checkout", "payments", "shared");
settings:dt.security_context IN ("checkout", "payments", "shared");
```

```
# Gen2 policy (transitional)
ALLOW environment:roles:viewer;
```

```
# Gen2 boundary
environment:management-zone IN ("checkout", "payments", "shared");
```

Wildcard Access

For admin groups needing full access (still two parallel policy bindings):

```
# Gen3 admin: all data + settings, no boundary scope
ALLOW storage:logs:read, storage:spans:read, storage:metrics:read,
      settings:objects:read, settings:objects:write;
```

```
# Gen2 admin: all Classic entity access (transitional, remove once MZ
retirement is complete)
ALLOW environment:roles:manage-settings, environment:roles:viewer;
```

> **Don't write `MATCH('*')` on a Gen2 policy.** `MATCH` is a Gen3-domain operator only; using it as a wildcard on `storage:entities:read` (a Gen2 surface) would silently grant access to all Gen2 entities.

Boundary Limitations

Limitation	Description	Workaround
Max 10 conditions	Only 10 lines per boundary	Create multiple boundaries
No AND between lines	Each line is OR-combined	Use multiple boundaries for AND logic
MATCH not in `environment:`	`MATCH()` not supported in `environment:` domain (Classic MZ)	Use `IN` (preferred) for management zones

4. Security Context and Data Partitioning Strategy

Security contexts and Grail buckets form the foundation of boundary filtering and data partitioning. Plan your strategy carefully.

> **Boundary design vs policy parameterization (read first).** This notebook covers **standalone Boundary resources** — separate IAM objects applied to groups for

> defense-in-depth scoping. Standalone boundaries are typically per-team with hardcoded scope (one boundary per team), distinct from the **policy WHERE clauses** in IAM-04 which ARE parameterizable via `${bindParam:NAME}` and bound per group. The two layer together: a parameterized policy (the "what" — bound per team) plus a per-team standalone boundary (the "where" — hardcoded per team) gives both scaling and defense-in-depth. See **IAM REFERENCE.md § Policy Parameterization and Boundary Standardization** for the canonical pattern; standardize the boundary FIELD on `dt.security_context` (Gen3) or management zone (Classic) regardless of which layer you're scoping.

What is a Security Context?

A **security context** is a label applied to:
- Entities (hosts, services)
- Data (logs, spans)
- Settings (configurations)

Boundaries filter based on these labels.

Primary Grail Fields

Primary Grail fields are first-class attributes that Dynatrace uses for data organization. These should drive your bucket, permission, and segmentation strategy:

Field	Source	Use Case
`k8s.cluster.name`	Kubernetes	Cluster-level isolation
`k8s.namespace.name`	Kubernetes	Namespace-level access
`aws.account.id`	AWS metadata	AWS account separation
`azure.subscription.id`	Azure metadata	Azure subscription separation
`gcp.project.id`	GCP metadata	GCP project separation
`dt.host_group.id`	OneAgent config	Host group-based isolation

These fields are automatically enriched from infrastructure metadata and provide consistent, reliable partitioning dimensions.

Grail Bucket Strategy

Buckets partition data in Grail for access control, retention, and query performance.

Key Principle: Set up buckets along organizational lines and route data based on primary Grail fields.

Bucket Design	Example	Benefit
By Environment	`prod_logs`, `dev_logs`	Separate prod/non-prod access
By Team/LOB	`checkout_data`, `payments_data`	Team-level isolation
By Compliance	`pci_logs`, `general_logs`	Regulatory separation
By Region	`us_data`, `eu_data`	Geographic compliance

Bucket Limitations

> ⚠️ **Critical:** Plan buckets carefully - these constraints cannot be changed later.

Limitation	Details
One data type per bucket	Logs, metrics, events, OR spans - not mixed
Names are immutable	Bucket names CANNOT be changed after creation
No data migration	Data CANNOT be moved between buckets
Naming rules	3-100 chars, lowercase alphanumeric, underscores, hyphens only
Maximum buckets	80 per environment (default limit)
Optimal ingest	~1 TB/day per bucket for best query performance
Acceptable ingest	1-3 TB/day per bucket (limited query window)
Maximum ingest	3 TB/day per bucket hard limit

Query Constraints

Limit	Value	Impact
Maximum data scanned	500 GB	Limits queryable time window
Maximum records returned	1,000	Use aggregations for larger datasets
Maximum response payload	1 MB	Large result sets may be truncated

Default Bucket Retentions

Bucket	Retention
`default_logs`	35 days
`default_metrics`	15 months
`default_spans`	10 days
`default_events`	35 days

Naming Convention: Include organization, data type, and retention in bucket names:
- `teamA_logs_90d`
- `prod_spans_30d`
- `pci_metrics_365d`

Bucket + Boundary Integration

> **When this combo fits:** the policy + standalone boundary pattern below is well-suited to scenarios where bucket-based scoping is already the right tool — compliance separation (PCI/HIPAA), retention isolation, or hard cost attribution. For general team-scoped data access without one of those reasons, a parameterized policy on `dt.security_context` alone (no bucket-name condition, no standalone boundary) is usually simpler. See **IAM REFERENCE.md § Bucket-Match Overlay** for scenario gating.

When the scenario does fit, the policy carries the **parameterized scope** and the boundary carries the **per-team hardcoded scope** for defense-in-depth. Pair them on the same group.

Policy (parameterized — one template, bound per team):

```
// Recommended: parameterized policy template
ALLOW storage:logs:read  WHERE storage:bucket-name = "${bindParam:log-
bucket}";
ALLOW storage:spans:read WHERE storage:bucket-name = "${bindParam:span-
bucket}";
```

Resolved when bound to the checkout group with `{"parameters": {"log-bucket": "checkout_logs", "span-bucket": "checkout_spans"}}`:

```
ALLOW storage:logs:read  WHERE storage:bucket-name = "checkout_logs";
ALLOW storage:spans:read WHERE storage:bucket-name = "checkout_spans";
```

Standalone Gen3 boundary (per-team — hardcoded scope is intentional here):

```
# Gen3 boundary — pair with a Gen3 policy (storage:*, settings:*)
# One boundary per team; standardize on dt.security_context as the universal
field.
storage:bucket-name IN ("checkout_logs", "checkout_spans");
storage:dt.security_context IN ("checkout");
settings:dt.security_context IN ("checkout");
```

Standalone Gen2 boundary (Classic — pair with a Gen2 policy, transitional):

```
# Gen2 boundary — management-zone is the Classic universal scoping field
environment:management-zone IN ("Checkout");
```

> **Why the policy is parameterized but the standalone boundary isn't:** policy WHERE clauses support `${bindParam:NAME}` and are bound per group at policy-binding time. Standalone Boundary resources are independent IAM objects — typically one boundary per team with hardcoded scope, attached to the group separately. See **IAM-10** for policy-binding mechanics; see **IAM REFERENCE.md § Anti-patterns** for why parameterizing on entity-type-specific fields (e.g., `host.name`) is the "dead in water" failure mode.

Using Buckets for Team Isolation

> **When this approach fits:** bucket-based team isolation is well-suited to scenarios where the bucket already exists for a specific reason — compliance separation (PCI/HIPAA), retention isolation, hard cost attribution, or hostile multi-tenancy. For general team-scoped access without one of those reasons, a parameterized policy on `dt.security_context` (no buckets, no standalone boundary) is usually simpler. The pattern below applies once you've decided buckets are the right tool. See **IAM REFERENCE.md § Bucket-Match Overlay** for scenario gating.

For team-level data access control (when the bucket scenario fits):

1. **Create team-specific buckets** - `teamA_logs`, `teamA_spans`, etc.
2. **Configure pipeline routing** - Route data to buckets based on primary Grail fields
3. **Create one parameterized policy** — bound to each team's group with different bucket parameters
4. **Create one standalone boundary per team** — hardcoded scope, applied per group

Recommended (parameterized policy + per-team standalone boundary):

```
// One parameterized policy template — bound per team
Policy: tpl-team-data-access (parameterized)
├── ALLOW storage:logs:read    WHERE storage:bucket-name = "${bindParam:log-
bucket}"
├── ALLOW storage:spans:read   WHERE storage:bucket-name = "${bindParam:span-
bucket}"
└── ALLOW storage:metrics:read WHERE storage:bucket-name =
"${bindParam:metric-bucket}"

// One standalone boundary per team — hardcoded scope (defense-in-depth)
Boundary: bnd-team-checkout
    storage:bucket-name IN ("checkout_logs", "checkout_spans",
"checkout_metrics");
    storage:dt.security_context IN ("checkout");
    environment:management-zone IN ("Checkout");

// Group: bind both — policy with team-specific parameters, boundary as-is
Group: Checkout-Team
├── Policy binding: tpl-team-data-access
│     parameters: {log-bucket: "checkout_logs", span-bucket:
"checkout_spans",
│                  metric-bucket: "checkout_metrics"}
└── Boundary attachment: bnd-team-checkout
```

Resolved view (what the policy evaluates to for the Checkout-Team group):

```
Group: Checkout-Team
├── Policy: tpl-team-data-access (resolved for this group)
│   ├── ALLOW storage:logs:read    WHERE storage:bucket-name =
"checkout_logs"
│   ├── ALLOW storage:spans:read   WHERE storage:bucket-name =
"checkout_spans"
│   └── ALLOW storage:metrics:read WHERE storage:bucket-name =
"checkout_metrics"
└── Boundary: bnd-team-checkout (unchanged — per-team hardcoded scope)
    storage:bucket-name IN ("checkout_logs", "checkout_spans",
"checkout_metrics");
    storage:dt.security_context IN ("checkout");
    environment:management-zone IN ("Checkout");
```

Adding a new team is one boundary YAML + one binding YAML — the policy stays unchanged. See **IAM-10: Templated Policy-Group Assignments** for binding mechanics and **IAM REFERENCE.md § Change-Management Caveat** for why parameter-shape decisions matter at design time.

> **For comprehensive bucket guidance**, see **ORGNZ-03: Bucket Strategy and Design** which covers naming conventions, retention planning, and cost attribution patterns.

Primary Grail Tags

Beyond fields, **Primary Grail Tags** can be configured from:
- Kubernetes labels (first 3 selected during setup)
- AWS resource tags
- Azure resource tags

These tags become first-class Grail attributes available for:
- Bucket assignment rules
- Policy/boundary conditions
- Pipeline routing
- Segment filters

Assigning Security Context

Security context is set via (modern approaches):

1. **Entity Enrichment rules** - Automatic based on entity properties
2. **Host properties** - Set on hosts and inherited by services
3. **Primary Grail fields/tags** - First-class tags for Grail data
4. **OneAgent group** - Inherited from deployment
5. **Log attributes** - Set during OpenPipeline ingestion

> **Note:** Auto-tagging is a legacy approach. Prefer Entity Enrichment and Primary Grail Fields for new implementations.

Security Context Design Patterns

Pattern	Example Values	Use Case
Team-based	`checkout-team`, `payments-team`	Team ownership
Application-based	`ecommerce-app`, `mobile-app`	App isolation

Pattern	Example Values	Use Case
Environment-based	`prod`, `staging`, `dev`	Env separation
Business unit	`retail`, `wholesale`, `corporate`	Business isolation
Geography	`us-east`, `eu-west`, `apac`	Regional separation

Naming Conventions

Rule	Good	Bad
Lowercase	`checkout`	`Checkout`
Hyphens	`team-checkout`	`team_checkout`
Descriptive	`payments-api`	`pa`
No spaces	`mobile-app`	`mobile app`

Shared Context

Some data should be visible to multiple teams:

```
Security Contexts:
├── checkout      (checkout team data)
├── payments      (payments team data)
└── shared        (cross-team data - infrastructure, common services)
```

Teams get their context + `shared`:

```
environment:management-zone IN ("checkout", "shared");
```

Recommended Partitioning Strategy

1. **Identify primary dimensions** - What drives access control? (team, environment, region, compliance)
2. **Map to Primary Grail Fields** - Use `k8s.cluster.name`, `aws.account.id`, `dt.host_group.id`
3. **Design buckets** - Create buckets aligned with access control boundaries
4. **Configure pipelines** - Route data to buckets based on primary fields

5. **Define policies** - Grant bucket-specific permissions per team
6. **Apply boundaries** - Use `storage:bucket-name` and `storage:dt.security_context`
7. **Create segments** - For cross-bucket query filtering

> **See Also:** For migration from Management Zones, refer to **MZ2POL-04: Policies and Boundaries** which covers mapping MZ patterns to the new model.

5. Common Boundary Patterns

Pattern 1: Single Team Boundary

Restrict to one team's data using two boundaries (Gen3 canonical + Gen2 transitional). Both attach to the same policy.

```
# Gen3 boundary — pair with Gen3 policy
storage:dt.security_context IN ("checkout");
settings:dt.security_context IN ("checkout");
```

```
# Gen2 boundary — pair with Gen2 policy (transitional)
environment:management-zone IN ("checkout");
```

Pattern 2: Team + Shared

Team data plus shared infrastructure:

```
# Gen3 boundary — pair with Gen3 policy
storage:dt.security_context IN ("checkout", "shared", "infrastructure");
settings:dt.security_context IN ("checkout", "shared");
```

```
# Gen2 boundary — pair with Gen2 policy (transitional)
environment:management-zone IN ("checkout", "shared", "infrastructure");
```

Pattern 3: Multiple Teams (Cross-Functional)

For SRE or platform teams:

```
# Gen3 boundary — pair with Gen3 policy
storage:dt.security_context IN ("checkout", "payments", "catalog", "shared");
settings:dt.security_context IN ("checkout", "payments", "catalog",
"shared");
```

```
# Gen2 boundary — pair with Gen2 policy (transitional)
environment:management-zone IN ("checkout", "payments", "catalog", "shared");
```

Pattern 4: Environment Tier

Production vs non-production:

Production (two boundaries):

```
# Gen3 boundary — pair with Gen3 policy
storage:dt.security_context IN ("prod-checkout", "prod-payments");
settings:dt.security_context IN ("prod-checkout", "prod-payments");
```

```
# Gen2 boundary — pair with Gen2 policy (transitional)
environment:management-zone IN ("prod-checkout", "prod-payments");
```

Non-Production (two boundaries):

```
# Gen3 boundary — pair with Gen3 policy
storage:dt.security_context IN ("dev-checkout", "staging-checkout");
settings:dt.security_context IN ("dev-checkout", "staging-checkout");
```

```
# Gen2 boundary — pair with Gen2 policy (transitional)
environment:management-zone IN ("dev-checkout", "staging-checkout");
```

Pattern 5: Read-All, Write-Scoped

Using two groups for the same user:

Group 1: All-Viewers (broad read)

```
Policy: environment-viewer
Boundary: environment:management-zone IN ("*");
```

Group 2: Checkout-Editors (scoped write)

```
Policy: checkout-write-policy
Boundary: environment:management-zone IN ("checkout");
```

Pattern 6: Application Team — Structured Security Context

When `dt.security_context` follows the `comp:<component>/bu:<bu>/app:<app>` format (see **Structured Security Context Design** in **IAM-04**), app teams can access all component types for their application using a single wildcard boundary:

```
// 3rd Gen Grail data — wildcard on app dimension (mid-string)
storage:dt.security_context MATCH('*/app:easytrade');

// Smartscape/Classic entities — anchored MATCH per component
// (do NOT use startsWith on storage:smartscape:read or storage:entities:read
— known bug)
storage:dt.security_context MATCH('comp:app/bu:digital/app:easytrade*');
storage:dt.security_context MATCH('comp:db/bu:digital/app:easytrade*');
storage:dt.security_context MATCH('comp:lb/bu:digital/app:easytrade*');
```

This grants access to all data tagged with any component prefix for that application:
- `comp:app/bu:digital/app:easytrade`
- `comp:db/bu:digital/app:easytrade`
- `comp:lb/bu:digital/app:easytrade`

Pattern 7: Transversal Team — Component-Based Access

Infrastructure teams (database, network, OS) that need cross-application access to their component type use a leading `comp:` prefix to enable a single rule:

```
// 3rd Gen Grail data — all database components, all applications
storage:dt.security_context MATCH('comp:db*');

// Smartscape/Classic entities — anchored MATCH (NOT startsWith — known bug)
storage:dt.security_context MATCH('comp:db*');
```

This grants access to all data matching:
- `comp:db/bu:digital/app:easytrade`
- `comp:db/bu:digital/app:easytravel`
- `comp:db/bu:corp/app:hipstershop`

> **Dimension order is the key design decision.** Placing `comp` first in the string means component-based transversal access is always a simple anchored match. Access by `bu` or other mid-string dimensions requires `MATCH('*/bu:digital/*')`.

Multi-Value Security Context (Upcoming)

> **Planned: CQ3** — `PRODUCT-14849`

The current single-string model requires choosing one leading dimension. A future **multi-value** `dt.security_context` will store each dimension independently:

```
dt.security_context = ["bu:digital", "app:easytrade", "comp:db"]
```

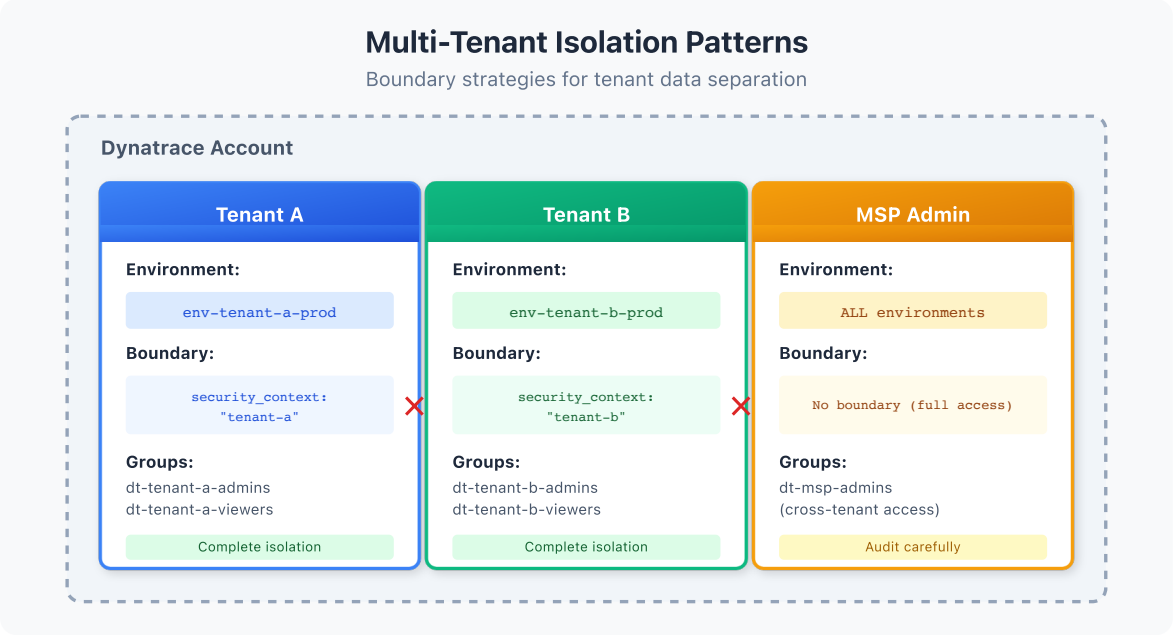
This removes the dimension-ordering trade-off and simplifies all three team types to a single boundary condition per signal type:

Team	Boundary (future)
App team (easytrade)	`storage:dt.security_context MATCH('app:easytrade')`
Database team (transversal)	`storage:dt.security_context MATCH('comp:db')`
Business unit (digital)	`storage:dt.security_context MATCH('bu:digital')`

Until multi-value is available, the structured single-string pattern above is the recommended approach.

6. Multi-Tenant Isolation

For organizations serving multiple customers or business units that require strict isolation.



Isolation Requirements

Requirement	Implementation
Data isolation	Unique security context per tenant
No cross-tenant access	Strict boundary matching
Audit capability	Centralized logging
Shared infrastructure	Separate "platform" context

Multi-Tenant Boundary Example

Tenant A Group (two boundaries):

```
# Gen3 boundary - pair with Gen3 policy
storage:dt.security_context IN ("tenant-a");
settings:dt.security_context IN ("tenant-a");
```

```
# Gen2 boundary - pair with Gen2 policy (transitional)
environment:management-zone IN ("tenant-a");
```

Tenant B Group (two boundaries):

```
# Gen3 boundary — pair with Gen3 policy
storage:dt.security_context IN ("tenant-b");
settings:dt.security_context IN ("tenant-b");
```

```
# Gen2 boundary — pair with Gen2 policy (transitional)
environment:management-zone IN ("tenant-b");
```

Platform Team (cross-tenant, two boundaries):

```
# Gen3 boundary — pair with Gen3 policy
storage:dt.security_context IN ("platform", "shared");
settings:dt.security_context IN ("platform", "shared");
```

```
# Gen2 boundary — pair with Gen2 policy (transitional)
environment:management-zone IN ("platform", "shared");
```

Entity Enrichment for Multi-Tenancy

Use Entity Enrichment rules (Settings > Entity Enrichment) to assign tenant context:

```
# Entity Enrichment rule based on host group
Rule: Assign security context from host group
Entity Type: Host
Condition: Host group name contains "tenant-"
Action: Set dt.security_context = {hostGroup.name}
```

Alternatively, use **host properties** to set context at deployment:

```
# Set via OneAgent installer
--set-host-property=dt.security_context=tenant-a
```

7. Boundary Testing

Verify boundaries work as expected before production use.

```
// Check security context distribution on services
fetch dt.entity.service
| summarize count = count(), by:{dt.security_context}
| sort count desc
| limit 20

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes SERVICE
// | summarize count = count(), by:{dt.security_context}
// | sort count desc
// | limit 20
```

```
// Find entities without security context (boundary gaps)
fetch dt.entity.service
| filter isNull(dt.security_context)
| fields entity.name, tags
| sort entity.name
| limit 50
```

```
// Check host security context coverage
fetch dt.entity.host
| summarize
    total = count(),
    withContext = countIf(isNotNull(dt.security_context)),
    missing = countIf(isNull(dt.security_context))
| fieldsAdd coveragePercent = round(100.0 * withContext / total, decimals: 2)

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes HOST
// | summarize
// total = count(),
// withContext = countIf(isNotNull(dt.security_context)),
// missing = countIf(isNull(dt.security_context))
// | fieldsAdd coveragePercent = round(100.0 * withContext / total, decimals:
2)
```

```
// Verify logs have security context
fetch logs, from: now() - 1h
| summarize
    total = count(),
    withContext = countIf(isNotNull(dt.security_context))
| fieldsAdd coveragePercent = round(100.0 * withContext / total, decimals: 2)
```

Testing Methodology

1. **Create test group** with the boundary
2. **Add test user** to the group
3. **Log in as test user** (or impersonate)
4. **Verify visible entities** match expected
5. **Verify hidden entities** are not visible
6. **Query data** to confirm filtering works

Validation Checklist

Test	Expected
List services	Only boundary-included services
Query logs	Only logs with matching context
View settings	Only settings with matching context
Access denied entity	Empty result or error

Next Steps

With boundaries configured, complete your IAM implementation:

Recommended Path

1. **IAM-06: User Lifecycle and Provisioning** - Automate user management
2. **IAM-07: Audit Logging and Compliance** - Monitor access patterns
3. **IAM-08: Multi-Environment IAM** - Scale across environments

Boundary Checklist

Before moving on, ensure you have:

- [ ] Understood the three-domain model
- [ ] Designed your security context strategy
- [ ] Created boundaries for each team/group

- [ ] Configured entity enrichment for context assignment
- [ ] Tested boundaries with real users
- [ ] Verified no entities are missing context

Summary

In this notebook, you learned:

- Boundary fundamentals and how they differ from policies
- The three-domain model (environment, storage, settings)
- Boundary syntax and operators
- Security context strategy and naming
- Five common boundary patterns
- Multi-tenant isolation design
- Boundary testing and validation

References

- Permission Boundaries
- Security Context
- Entity Enrichment
- Host Properties
